DATA STRUCTURES USING C

Made By : **KASHIF SAYYAD**

MODULE I — DATA STRUCTURE AND ALGORITHM ANALYSIS

■ Data Structures — Introduction to Linear and Non-linear Data Structures

Data is useless until it is arranged in a particular way. A **data structure** is a logical way of organising, processing, retrieving and storing data in the memory of the computer so that operations like insertion, deletion, search and traversal become easy and efficient. The choice of a data structure decides the simplicity of an algorithm and its running time.

Linear data structures arrange the elements at one level so that every element has one predecessor and one successor: **arrays, linked lists, stacks and queues**. **Non-linear data structures** arrange the elements hierarchically or in networks: **trees** (one root, parent-child) and **graphs** (arbitrary connections). There is also the division of **homogeneous / non-homogeneous** structures and of **static (array)** vs **dynamic (linked list)** structures.

> **Remember :** Abstract Data Type (ADT) is a mathematical model in which a set of values and a set of operations are defined without showing the internal implementation (like the black box view). Stack, Queue and List are the classic ADTs.

■ Algorithm Analysis — Growth Rates, Big-O Notation

An **algorithm** is a finite sequence of well-defined steps to solve a problem. **Algorithm analysis** studies the behaviour of an algorithm by measuring how much **time** and how much **space** it needs when the size of input (n) grows. **Growth rate** means how fast the cost function increases with n. The asymptotic notations used are: Big-O (upper bound), Omega (lower bound) and Theta (both-bounds tight). Big-O notation of the syllabus: $f(n) = O(g(n))$ when some constant c and n_0 exist so that $f(n) \leq c \cdot g(n)$ for all $n > n_0$.

Common Big-O classes	**Typical algorithm**
O(1) — constant	accessing array element, stack push / pop
O(log n) — logarithmic	binary search
O(n) — linear	linear search, traversal of a list
O(n log n)	merge sort, quick sort (average), heap sort
$O(n^2)$ — quadratic	bubble sort, selection sort, insertion sort
$O(n^3)$	matrix multiplication (simple)

$O(2^n)$	exponential brute-force algorithms

The analysis of every algorithm is considered in three cases: **best case** (minimum time), **average case** (expected over all inputs) and **worst case** (maximum over all inputs; most used for practical guarantees).

MODULE II — ARRAYS AND LINEAR ARRAYS

■ Need of Arrays, Linear Arrays

C-style variables can hold only one value at a time; to store 100 marks of a class, 100 separate variables would be unmanageable. This is the **need of arrays**: an array keeps many values of the same type under a single name. A **linear array** (one-dimensional array) is a list of a finite number n of similar data elements referenced respectively by an index number.

Representation of a linear array in memory: the elements are stored in **consecutive (contiguous) memory locations**. The address of the first element is called the **base address**, and the address of the ith element is computed by the formula.

$$Address(a[i]) = Base + (i - LB) \times sizeof(element) \quad (LB = \text{lower bound, usually } 0)$$

■ Row-Major and Column-Major, Operations on Linear Arrays

Row-major order stores the matrix keeping the rows contiguous (row 1 first, then row 2 ...), whereas **column-major order** keeps the columns contiguous (column 1 first, then column 2 ...). The two are the standard memory orders of a 2-D array.

Operations on linear arrays:

1. **Traversing** — visiting each element of the array exactly once to process the whole data: done simply by a loop from lower bound to upper bound, cost O(n).
2. **Insertion** — putting a new element at position k: all the elements from position k onwards are shifted one place to the right, cost O(n) worst case.
3. **Modification** — replacing the value of an element at a known index a[k] = newvalue; direct access, cost O(1).
4. **Deletion** — removing the element at position k and shifting the remaining elements one place to the left, cost O(n).
5. **Merging** — joining two sorted arrays A and B into a single sorted array C: compare the current elements of A and B, copy the smaller and advance; O(m + n).

■ Two Dimensional Arrays and Their Representation, Sparse Matrices

A **2-D array** (matrix) with m rows and n columns is declared in C as int a[m][n]. Since memory is strictly one-dimensional, the 2-D array must be mapped linearly: if **row-major** is used, address(a[i][j]) = base + (i − $LB)_{row}$×n + (j − $LB)_{col}$) × size; for **column-major**, the roles of i and j are interchanged. Three-dimensional and higher arrays are stored with the same principle, first varying the right-most subscript.

A **sparse matrix** is a matrix in which most of the elements are zero. Storing all m×n cells wastes space, so a sparse matrix is represented by keeping only the **non-zero elements** as triples (row, column, value), usually in a list. This saves storage and speeds up operations like transpose and addition.

■ Searching and Sorting, Insertion Sort

■ Searching — Linear Search and Binary Search

Linear (sequential) search starts from the first element and compares the key with every element one by one until it is found or the list ends. It works on unsorted data as well, and its complexity is O(n) in the worst case.

Binary search halves the interval at every step: compare the key with the middle element; if smaller, search the left half; if bigger, search the right half; repeat until found or interval empty. It needs a **sorted array** and its complexity is O(log n). It is among the most important searching algorithms.

■ Sorting — Bubble Sort, Selection Sort

Bubble sort repeatedly compares the neighbouring elements and swaps them if they are in the wrong order; after the first pass the largest element reaches last position (bubble). It runs n−1 passes and the complexity is $O(n^2)$.

Selection sort in every pass finds the minimum of the unsorted part, and puts it in the position where the pass starts; after n-1 passes the array becomes sorted. The total of comparisons is fixed at n(n−1)/2; complexity $O(n^2)$.

Insertion sort builds the sorted list one element at a time: take the current key, shift all larger elements of the sorted part one step right and put the key into its correct place. It is $O(n^2)$ worst case but very efficient for nearly-sorted arrays and small n; it is also an online algorithm that sorts as elements arrive.

MODULE III — STACK AND QUEUE

■ Stack — Introduction and Operations

A **stack** is an ordered list in which insertion and deletion are performed at one end only, called the **top**. Because the last inserted element is removed first, a stack is called a **LIFO (Last In First Out)** structure. The **operations on stack** are: **push(x)** (insert x after checking overflow; top++), **pop()** (delete top element after checking underflow), **peek / top()** (only reads the top element without removal), **isEmpty()** and **isFull()**.

Stack implementation using arrays uses an array and an integer top that starts from −1 (empty). Push first checks top == SIZE−1 (overflow) and in case of no overflow stores the element at top. Pop first checks top == −1 (underflow); then takes the top element and does top−−. Both push and pop take O(1) time.

Remember : Infix expressions, recursion, undo operations, and system function calls are internally all stacks.

■ Applications of Stack — Expression Representation and Evaluation

The position of the operator relative to the operands decides the **notation of an expression**: **infix** (operator between operands, e.g. A + B), **prefix or Polish** (operator before, e.g. + A B), and **postfix or reverse Polish** (operator after, e.g. A B +). Compilers transform infix into postfix because the postfix form can be evaluated in one scan without any parenthesis and priority rules.

Conversion of infix to postfix is done with a stack: operands are copied directly to output; an operator is pushed only after popping and outputting all operators of higher or equal priority from the stack; '(' is pushed and ')' pops out operators until the '(' is reached; after the scan the stack is emptied out.

Expression evaluation of a postfix expression is also stack based: scan from the left, push the operands; on meeting an operator, pop two operands, apply the operator to them, push the single result back; the last value in the stack is the value of the expression. æ ˆ evaluation of parenthesised and local variable expressions and of unary / binary and ternary operations is all in stack form.

■ Queue — Introduction, Types and Operations

A **queue** is an ordered list in which insertion is done at one end (**rear**) and deletion at the other end (**front**); because the first inserted element is removed first, it is a **FIFO (First In First Out)** structure. Queue operations are enqueue (insertion at rear), dequeue (deletion from front), with checks of overflow and underflow.

Types of queues: simple linear queue; **circular queue**, in which the rear (and front) wrap around to the beginning once the end of the array is reached using (rear + 1) % SIZE — this removes the false-overflow of a plain queue and all the cells can be reused; **dequeue (double-ended queue)**, which allows insertion and deletion at both ends; and the **priority queue**, in which every element carries a priority and the element with the highest priority is served first even if it entered last.

Queue implementation using arrays keeps the current indices front (starts 0), rear (starts −1), and size; enqueue checks full, rear++ then store; dequeue checks empty, front++ then remove. Total cost of both O(1). **Operations on queue**: traversing from front to rear, insertion, deletion, modification (change of an element's data or priority), and the count of elements.
